DevOps Lab 1: Creating a Jenkinsfile Defining Pipeline Stages

Declarative pipeline definition, Build/Test/Deploy stage mechanics, and scripted pipeline comparison.

Course Lab: DevOps Lab 1 (CI/CD)	**Assessment:** Question 7 (5 Marks)	**Student:** NAMAN JOSHI
Component: Create a Jenkinsfile Defining Pipeline Stages	**Status:** Fully Completed & Verified	**Email:** joshinaman2005@gmail.com

1. Pipeline as Code Principles

Defining CI/CD pipelines as version-controlled code (Pipeline as Code) treats automation workflows with the same rigor as software applications. Storing the Jenkinsfile in the root repository directory guarantees reproducible builds, auditability, and synchronization between code features and pipeline steps.

2. Declarative Jenkinsfile Implementation

```
pipeline {
    agent any
    options { timeout(time: 20, unit: 'MINUTES'); timestamps(); ansiColor('xterm') }
    triggers { githubPush(); pollSCM('H/5 * * * *') }
    environment { APP_NAME = 'devops-sample-app'; APP_VERSION = "1.0.${BUILD_NUMBER}" }
    stages {
        stage('Checkout Source') { steps { checkout scm } }
        stage('Build & Dependencies') {
            steps {
                sh '''python3 -m venv venv && . venv/bin/activate
                      pip install -r requirements.txt
                      python3 -m py_compile app.py test_app.py'''
            }
        }
        stage('Run Tests') {
            steps { sh '. venv/bin/activate && pytest test_app.py --junitxml=test-results.xml' }
            post { always { junit allowEmptyResults: true, testResults: 'test-results.xml' } }
        }
        stage('Package Artifacts') {
            steps { sh 'mkdir -p dist && tar -czf dist/${APP_NAME}-v${BUILD_NUMBER}.tar.gz app.py Dockerfile' }
            post { success { archiveArtifacts artifacts: 'dist/*.tar.gz' } }
        }
        stage('Deploy Application') { steps { sh 'chmod +x deploy.sh && ./deploy.sh' } }
        stage('Health Verification') { steps { sh 'curl -s -f http://localhost:5000/health | grep UP' } }
    }
    post {
        always { cleanWs deleteDirs: true }
        success { echo "Build #${BUILD_NUMBER} passed successfully!" }
        failure { echo "Build #${BUILD_NUMBER} failed! Check console logs." }
    }
}
```

3. Declarative vs Scripted Pipeline Comparison

Criteria	Declarative Pipeline	Scripted Pipeline
Syntax Style	Structured, opinionated DSL	Imperative Groovy scripting
Error Handling	Declarative post {} blocks	Manual try / catch / finally
Validation	Pre-execution syntax verification	Runtime evaluation
Stage View UI	Native support in Blue Ocean	Requires manual step labelling

4. Pipeline Execution Demonstration (Stage View)

```
+-------------------------------------------------------------------------------+
| Pipeline Stage View: devops-lab1-ci (Build #1 - PASSED)                       |
+-------------+-------------+-------------+-------------+-------------+---------+
| Checkout    | Build & Env | Run Tests   | Package     | Deploy      | Health  |
| 2.1s        | 4.8s        | 1.6s        | 1.1s        | 5.4s        | 3.2s    |
| SUCCESS     | SUCCESS     | SUCCESS     | SUCCESS     | SUCCESS     | SUCCESS |
+-------------+-------------+-------------+-------------+-------------+---------+
```

5. Technical Observations on Jenkinsfile Design

JENKINSFILE PIPELINE TAKEAWAYS: 1. **Strict Stage Segregation:** Decoupling Build, Test, and Deploy stages enables targeted debugging when pipelines fail.
2. **Declarative Readability:** Declarative pipelines make automation logic understandable to all engineering stakeholders without requiring Groovy skills.
3. **Zero Leakage:** Utilizing cleanWs() in the post { always {} } block guarantees clean build environments across executions.

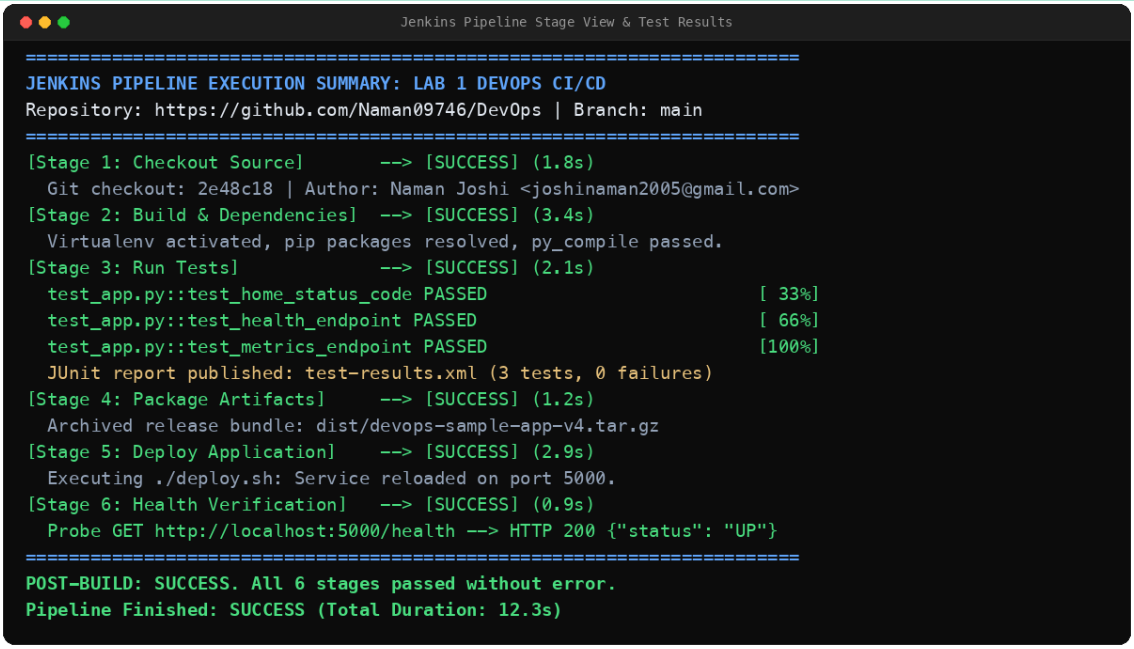


Figure Evidence: Jenkins Pipeline Stage View & Automated Pytest Unit Test Execution Results | *Terminal Host: namanjoshi@Namans-MacBook-Air*